

G7 - CAGT RECALL AI

Com
Ao
Gao
Tien



recall.ai

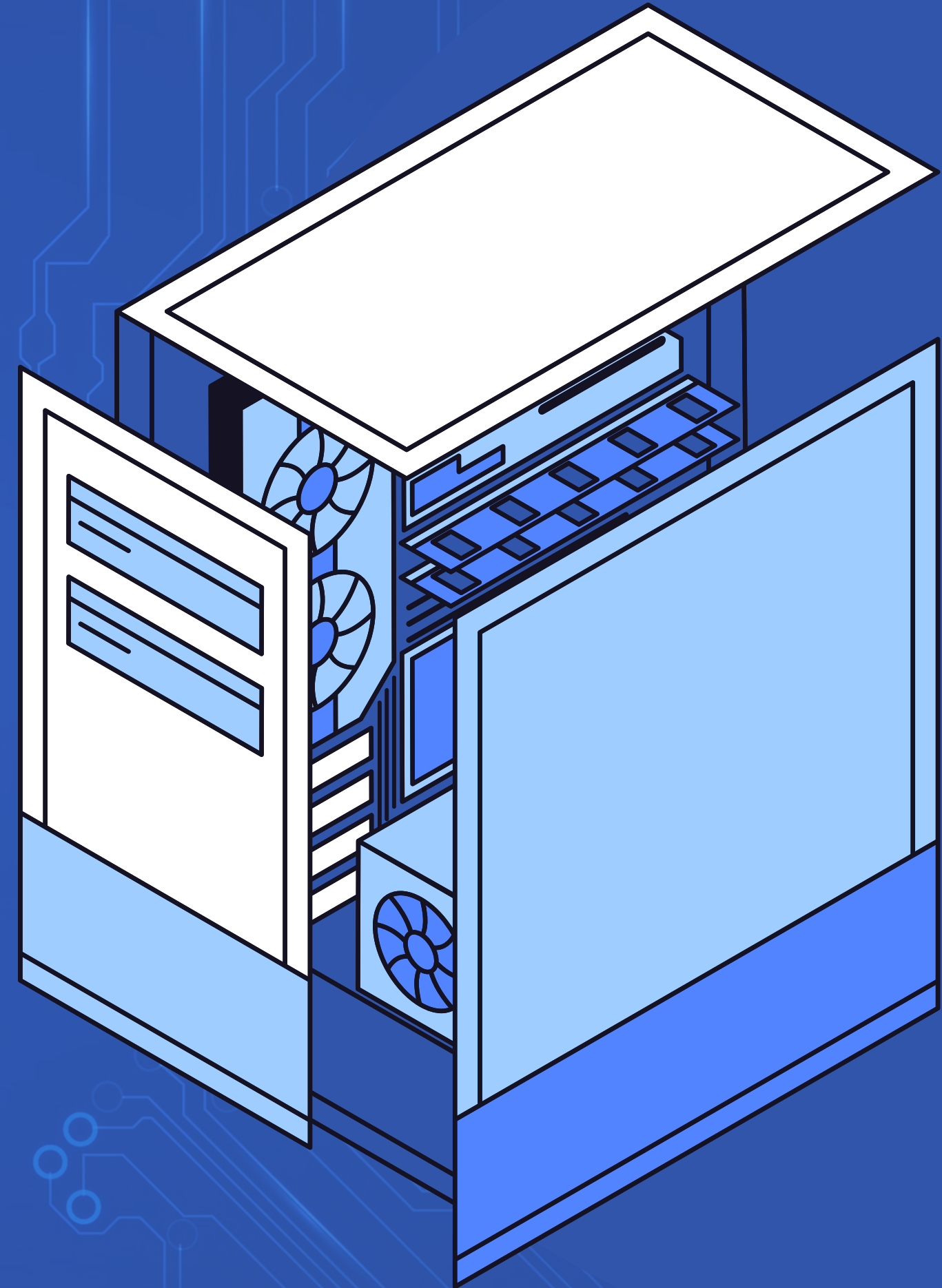
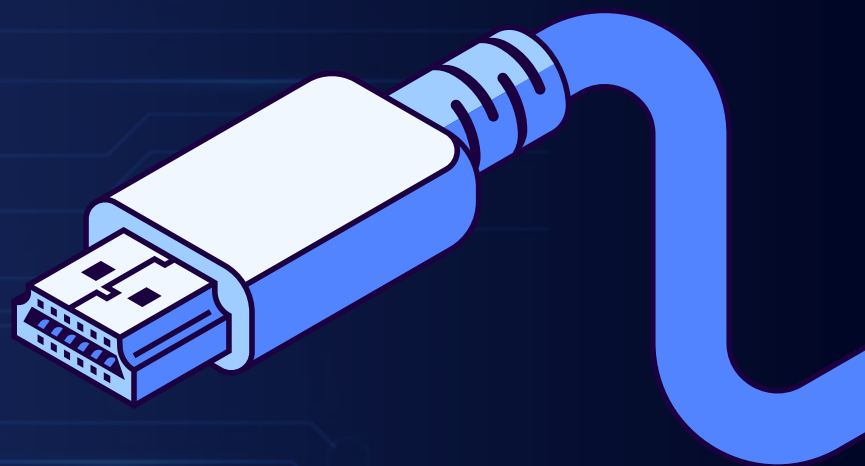
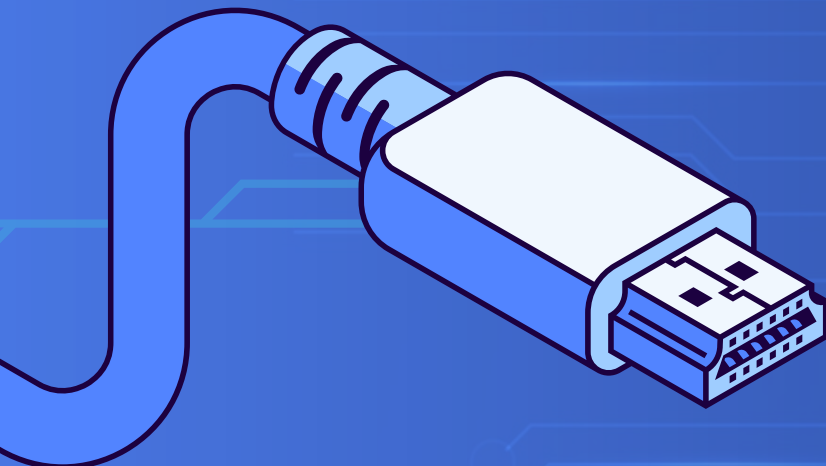




TABLE OF CONTENT

Title	01.	Project Management	07.	System architecture	13.
Table of content	02.	Use case model	08.	Technologies used	14.
Team Members	03.	UC - Study planner	09.	Software testing	15.
Problem statement	04.	UC - Focus session	10.	Key scenario	16.
Product position	05.	UC - AI examiner	11.	Q&A	17.
Users	06.	Non-func requirements	12.	Thank you.	18.



TEAM MEMBERS

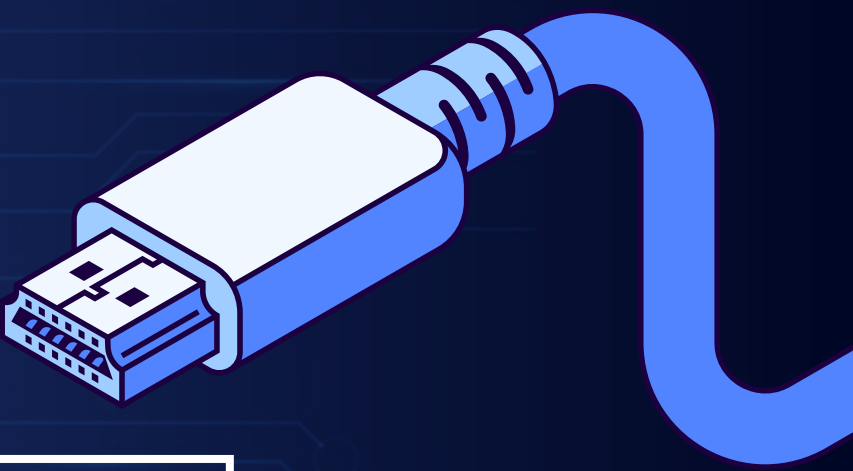
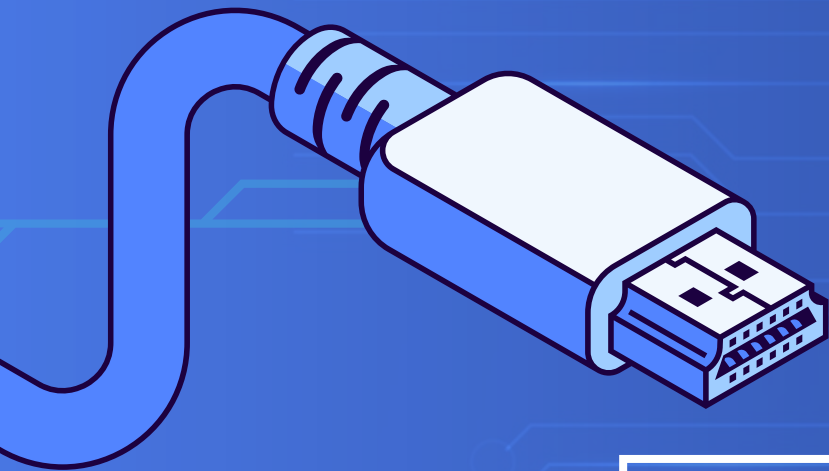
Thai Nguyen Tuan Kiet *Project Manager + UI/UX Designer*

Nguyen The Quan *Architecture + Fullstack Developer*

Nguyen Minh Phat *Tester + Fullstack Developer*

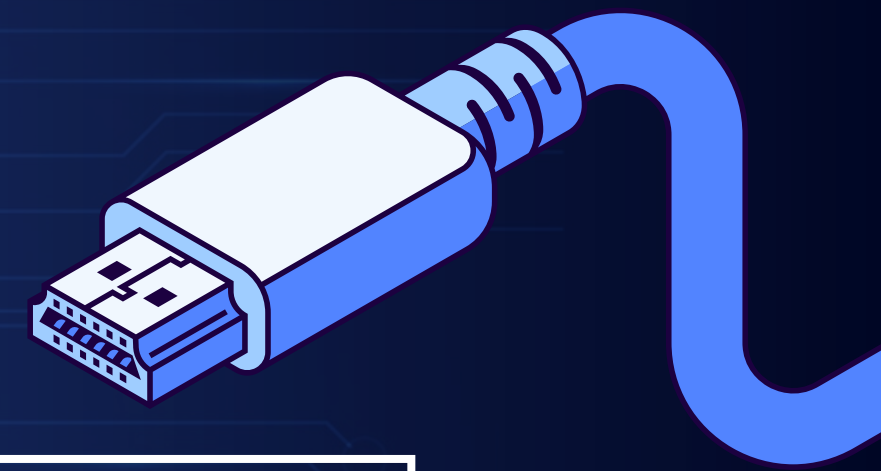
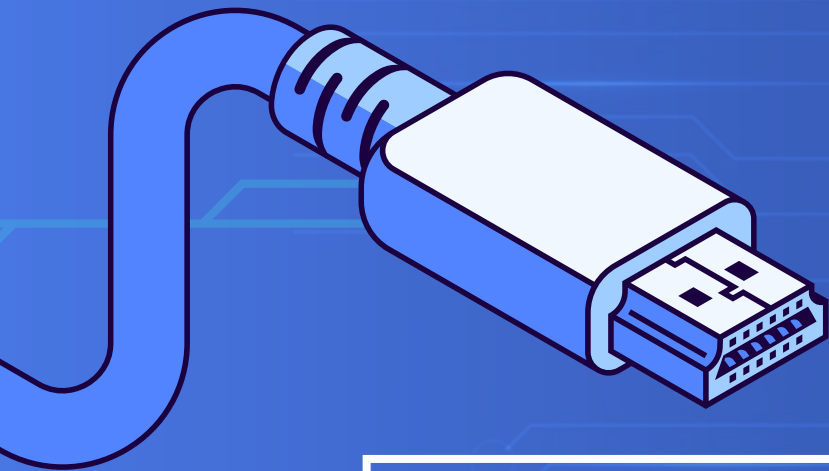
Nguyen Phuong Gia Bao *Frontend Lead*

Ngo Van Phong *Backend Lead*



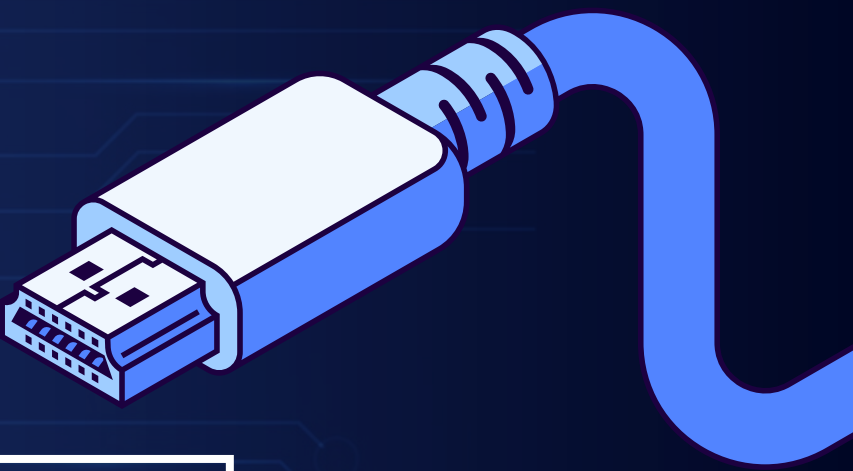
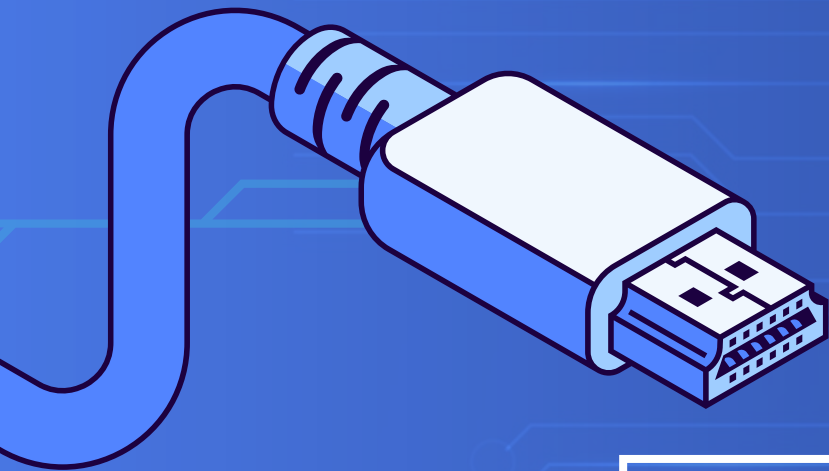
PROBLEM STATEMENT

Problem	Affect	Impact	Solution
• Passive learning • Lack of objective self-assessment • Unstructured cramming before exams • No way to trace gaps to root concepts	• 1st–2nd year students • Engineering and natural sciences majors • Theory-heavy, hierarchical-concept subjects	• Low memorization efficiency • False sense of understanding (familiarity ≠ mastery) • Ineffective revision • Repeating mistakes, root gaps unresolved	• AI-powered active revision assistant • Auto-generates structured study plans from materials • Multi-turn interviews to verify true understanding • Concept graph to trace and remediate knowledge gaps



PRODUCT POSITION STATEMENT

For	Who	The Recall AI	That	Unlike	Our product
1st-2nd year engineering & science students	Struggle with passive learning and spotting knowledge gaps	An AI-powered active revision assistant	Auto generates study plans and verifies understanding via AI interviews	NotebookLM , Quizlet, RemNote	Traces wrong answers to root concepts via a Concept Graph Engine



USERS

Name	Description	Responsibilities
University Student	1st and 2nd-year university students in engineering and natural sciences studying subjects with heavy theory and hierarchical concepts.	Uploads study materials, sets revision deadlines, participates in multi-turn AI interviews, and tracks their learning progress and knowledge gaps via the concept graph dashboard.



PROJECT MANAGEMENT

Thai Nguyen Tuan Kiet

Project Manager + UI/UX Designer

Nguyen The Quan

Architecture + Fullstack Developer

Nguyen Minh Phat

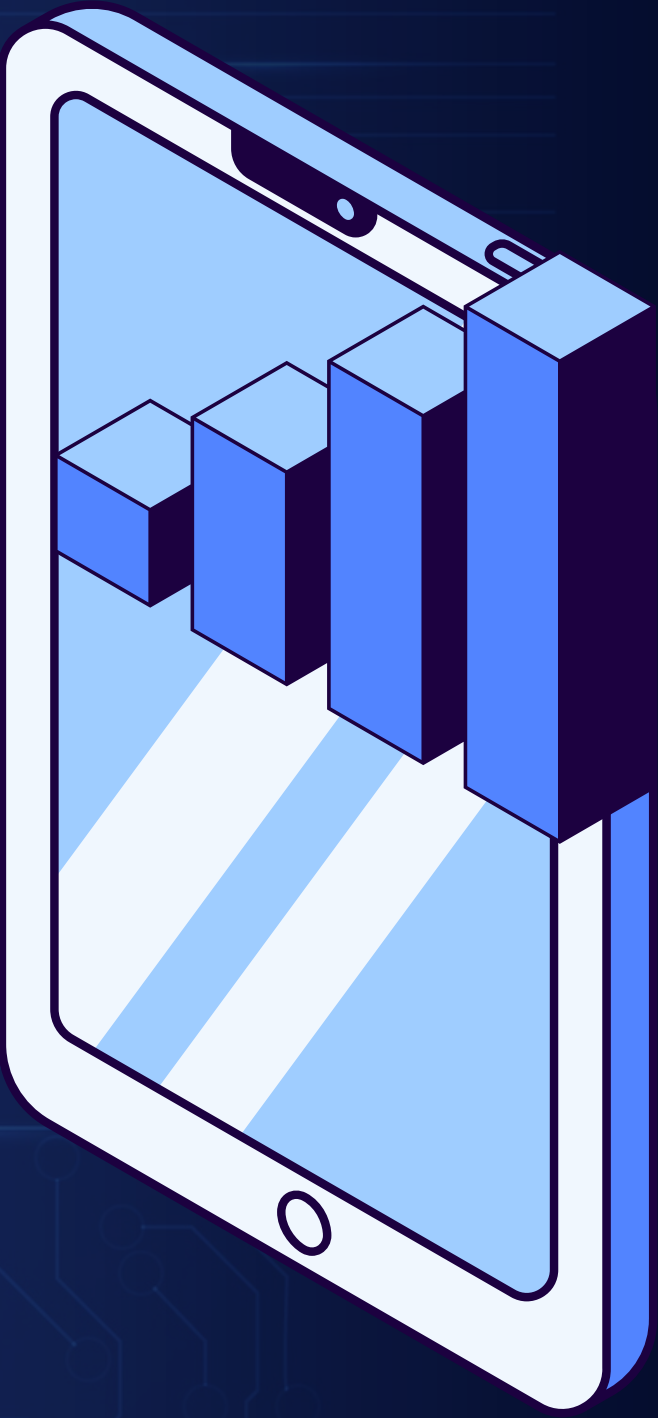
Tester + Fullstack Developer

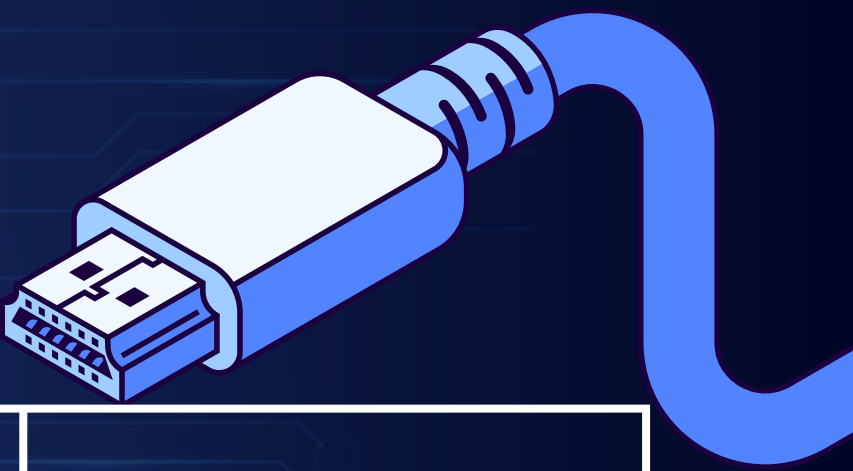
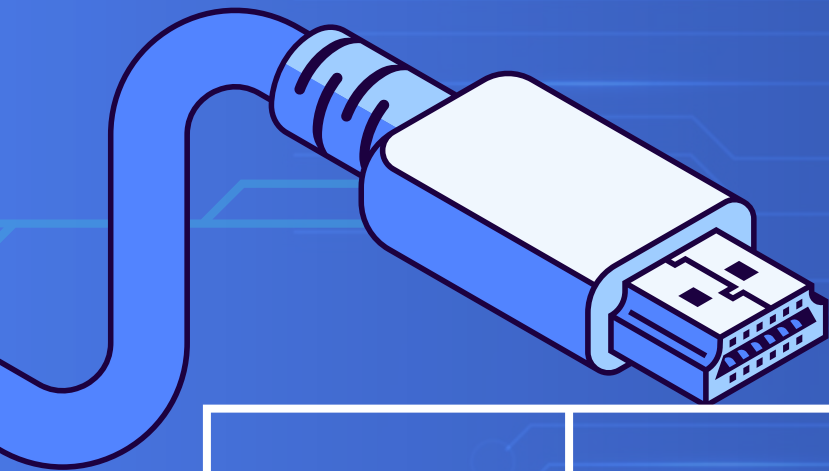
Nguyen Phuong Gia Bao

Frontend Lead

Ngo Van Phong

Backend Lead



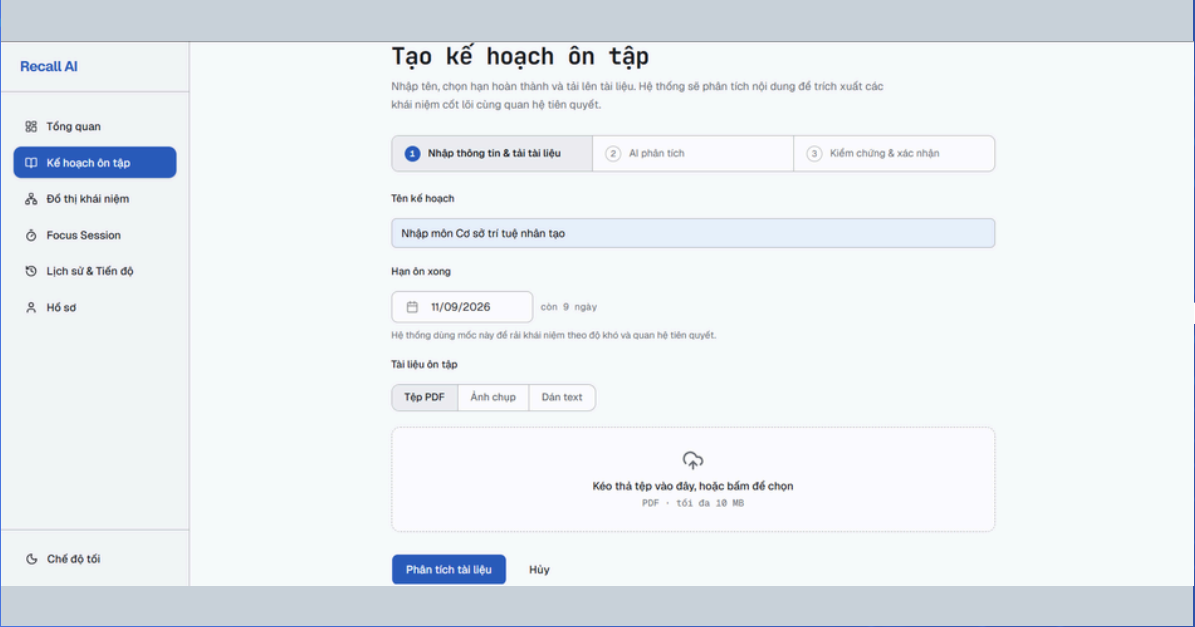


USE CASE MODEL

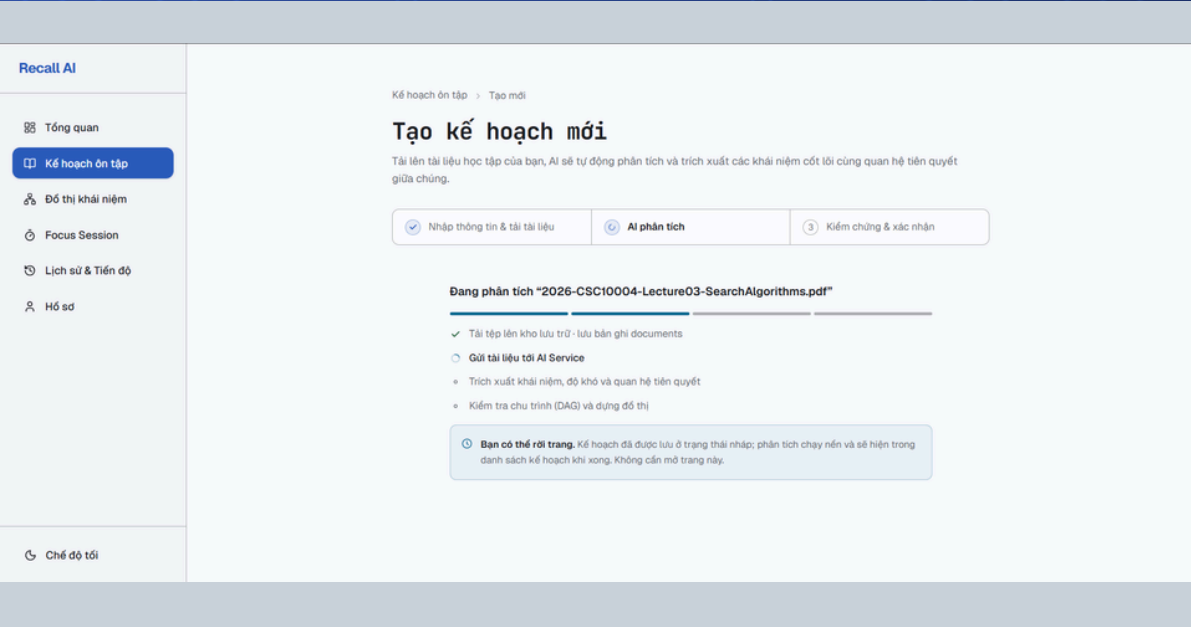
UC	Account	Study Planner	Focus Session	AI Examiner	Dashboard
Actor	Student	Student, AI Service, Scheduling & Remediation Engine	Student, System	Student, AI Service, Scheduling & Remediation Engine	Student, Scheduling & Remediation Engine
Goal	Authenticate account to use the system	Transform raw study material into a structured learning plan with a concept graph	Record focused study time (Pomodoro or free timer) linked to specific concepts	Objectively verify student's understanding via adaptive questioning; update mastery scores	Give students a view of their overall learning progress and today's priority concepts



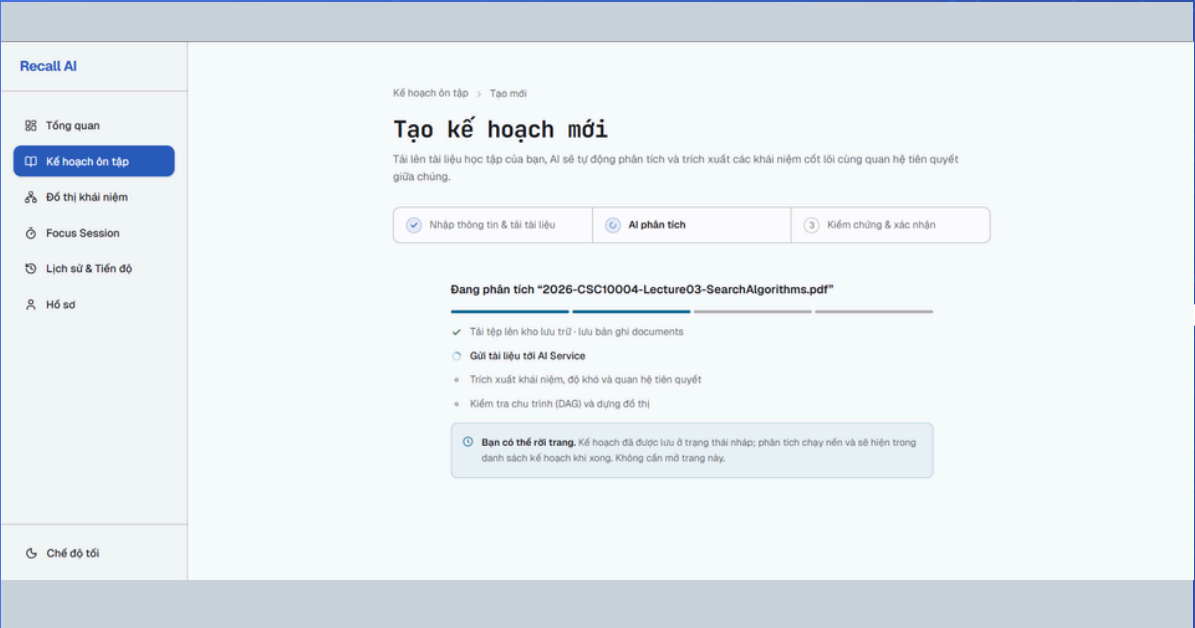
UC – STUDY PLANNER



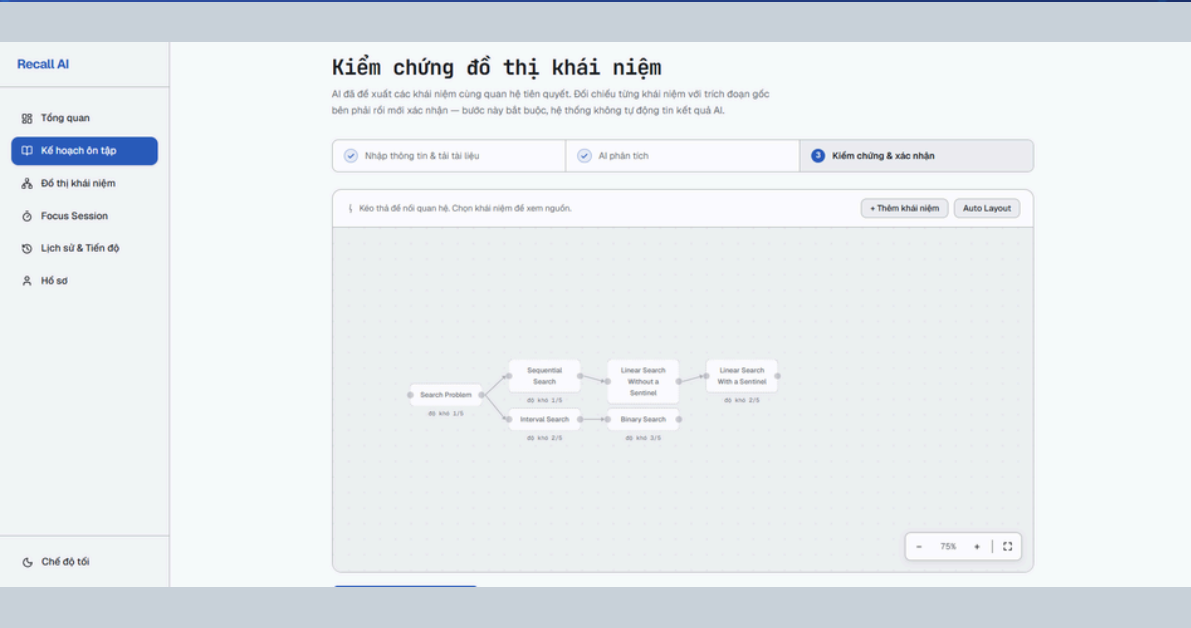
Student uploads document (PDF / image / paste text)



AI extracts concepts + prerequisite relationships → returns structured JSON



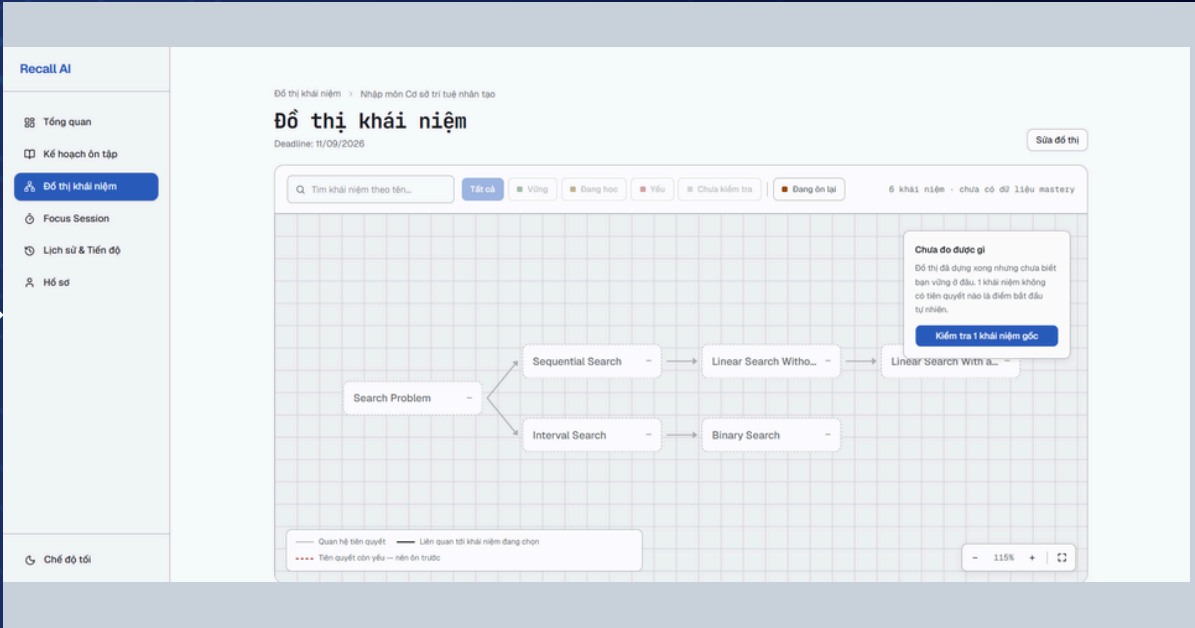
System validates as a DAG (no cycles)



Student reviews & edits the concept graph (react-flow)

KEY BEHAVIORS

- AI auto-retries on invalid output; cycles are removed automatically
- Draft is persisted before AI call to prevent data loss on exit



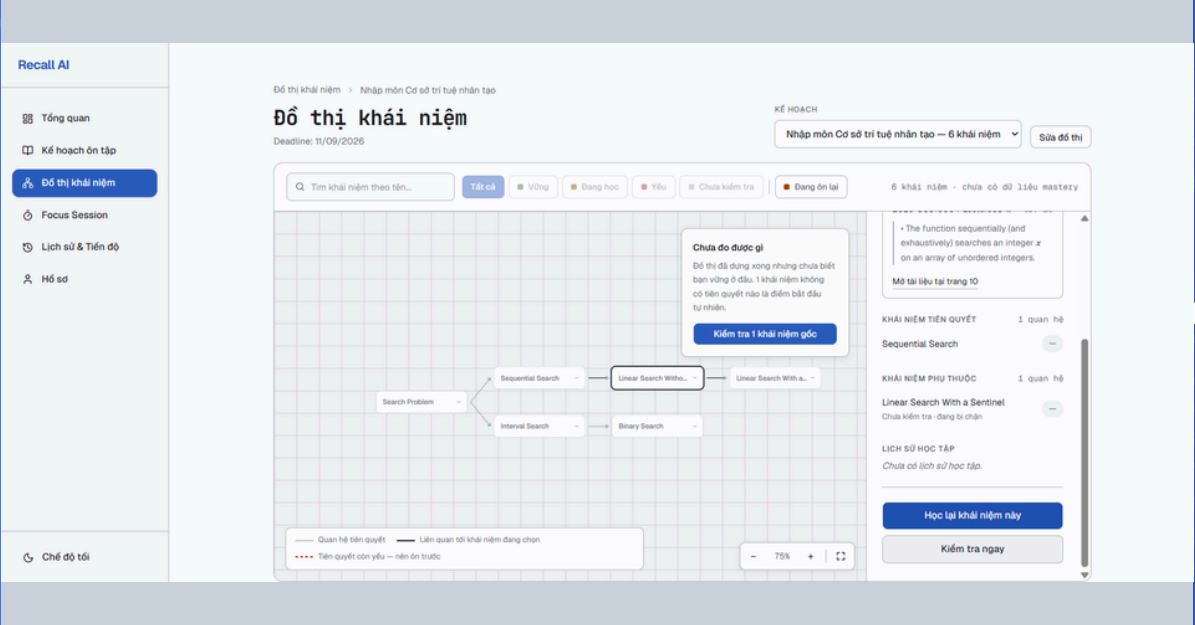
Confirm → plan saved, initial review schedule auto-generated



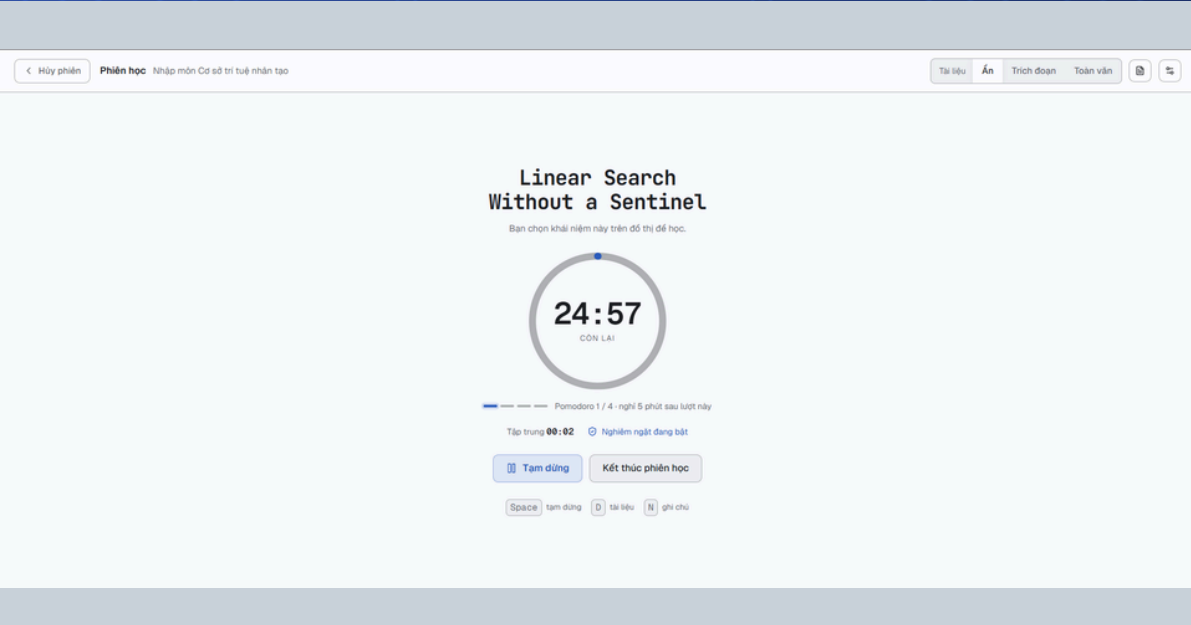
UC – FOUCS SESSION

KEY BEHAVIORS

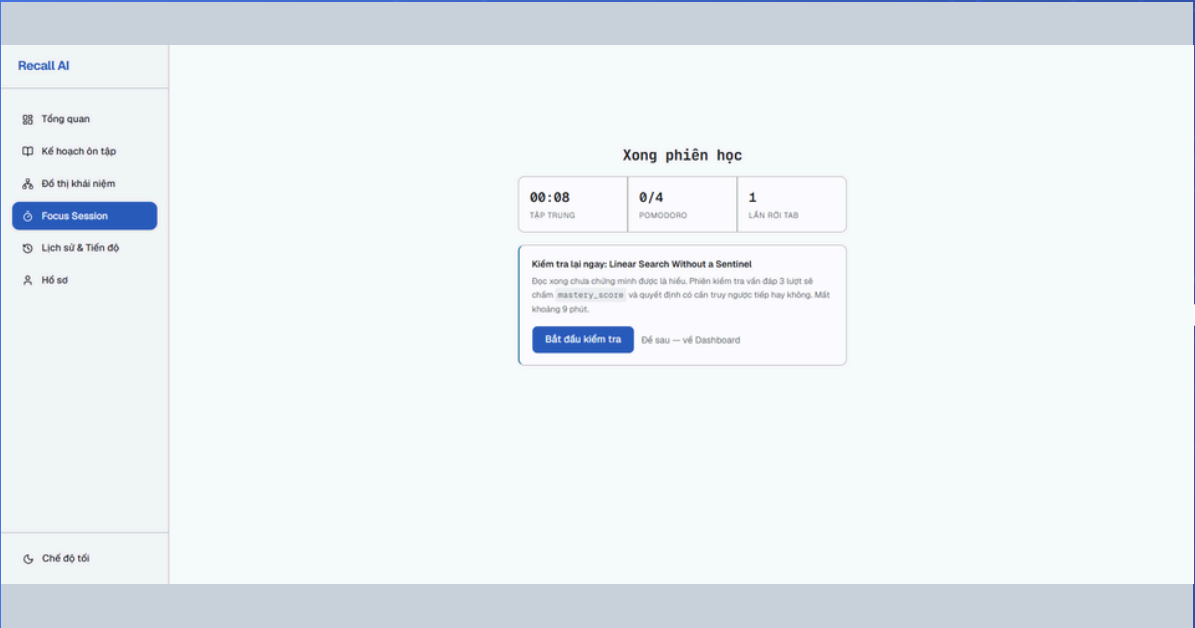
- Pause/resume supported; actual time recorded if ended early
- Timer runs client-side - works offline, syncs when reconnected



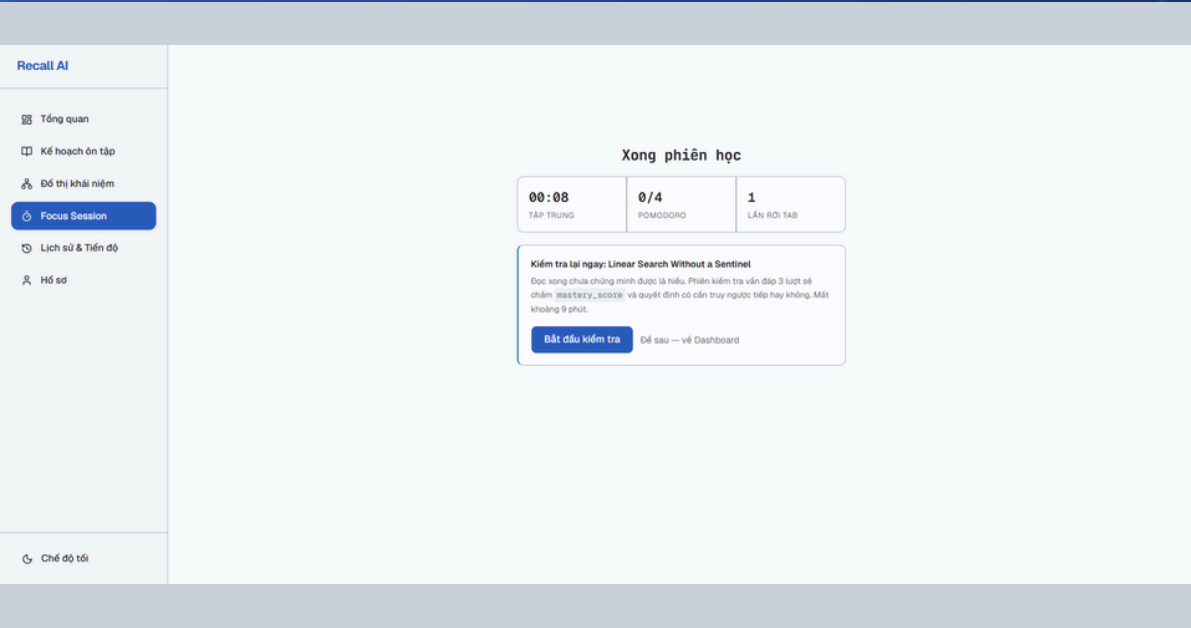
Student selects concepts to study → starts timer (default: 25 min Pomodoro)



Timer runs; student studies independently



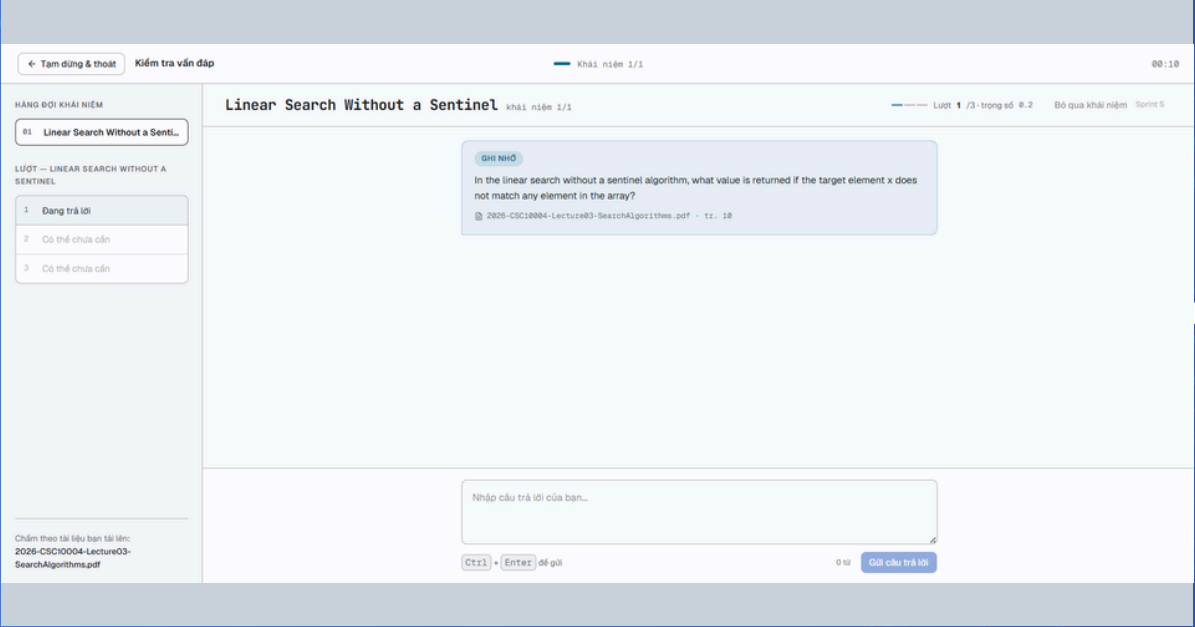
Session ends → system logs {user, concepts, start, end, duration}



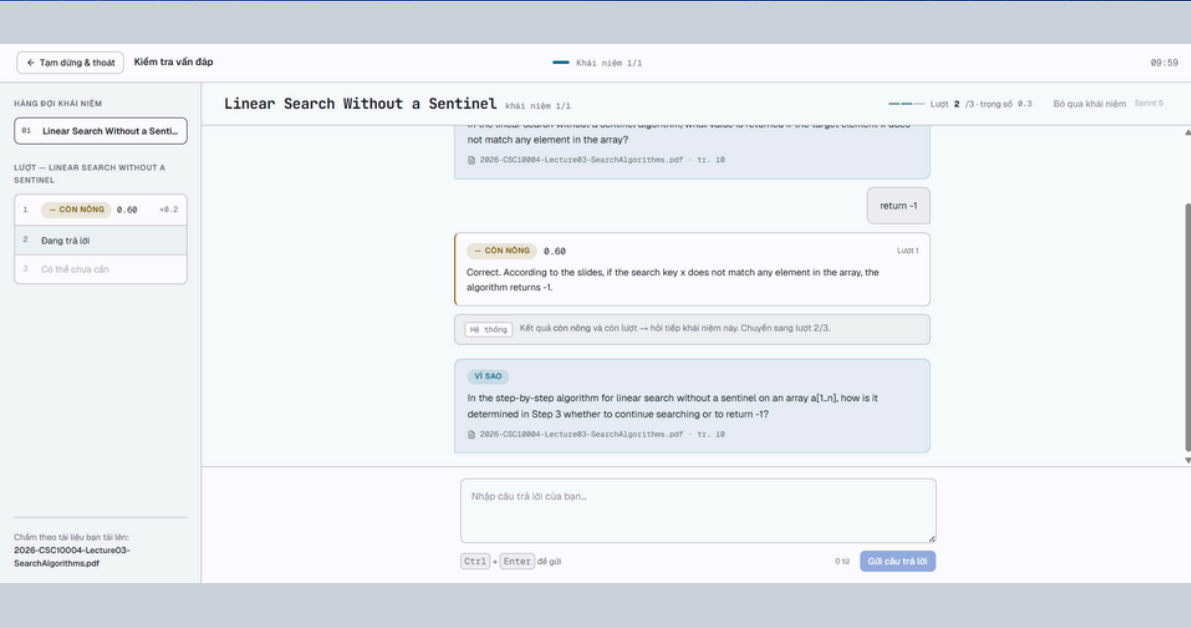
Prompt: "Start an Interview session now?"



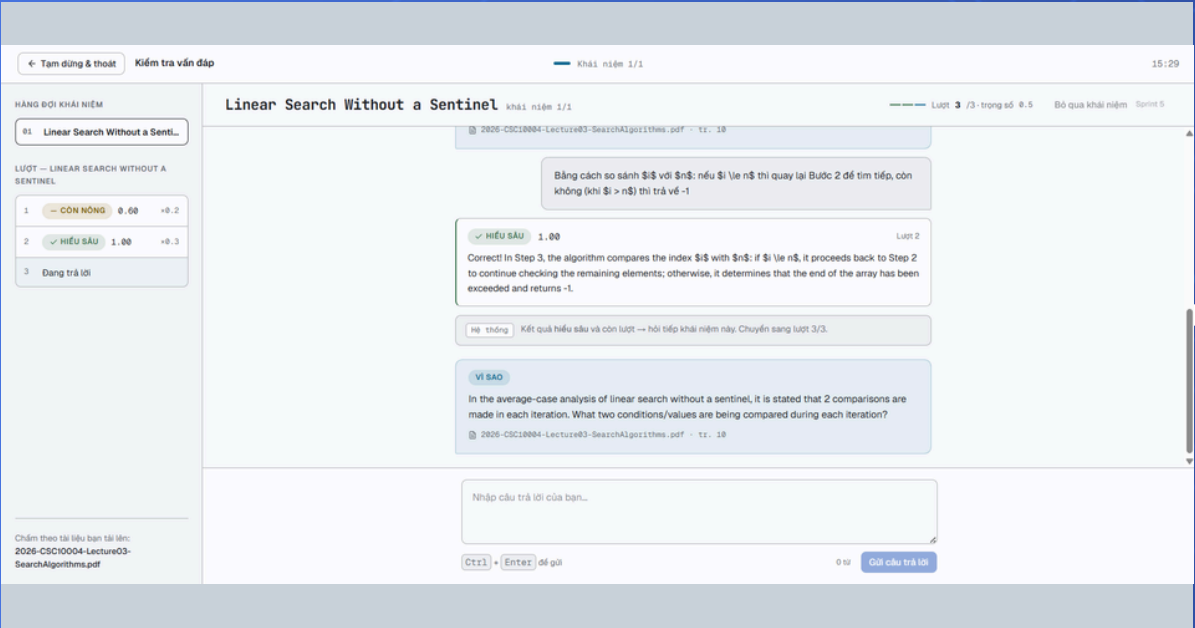
UC – AI EXAMINER



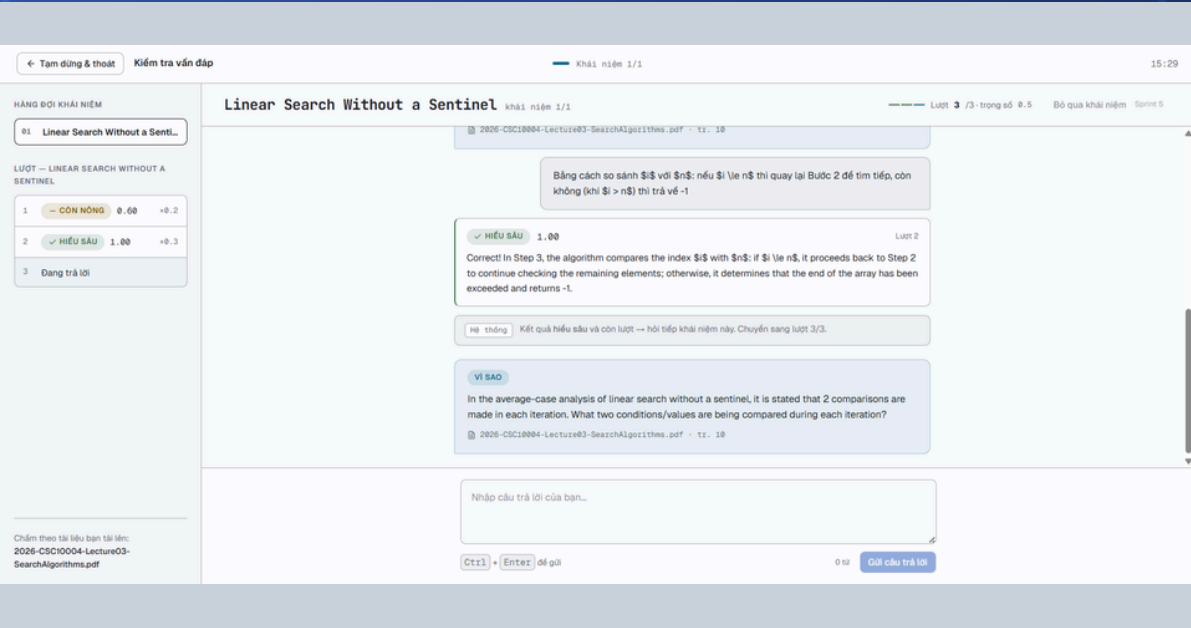
AI generates a question for concept C (recall / application / why)



Student answers → AI grades (score 0–1, verdict: deep / shallow / wrong)



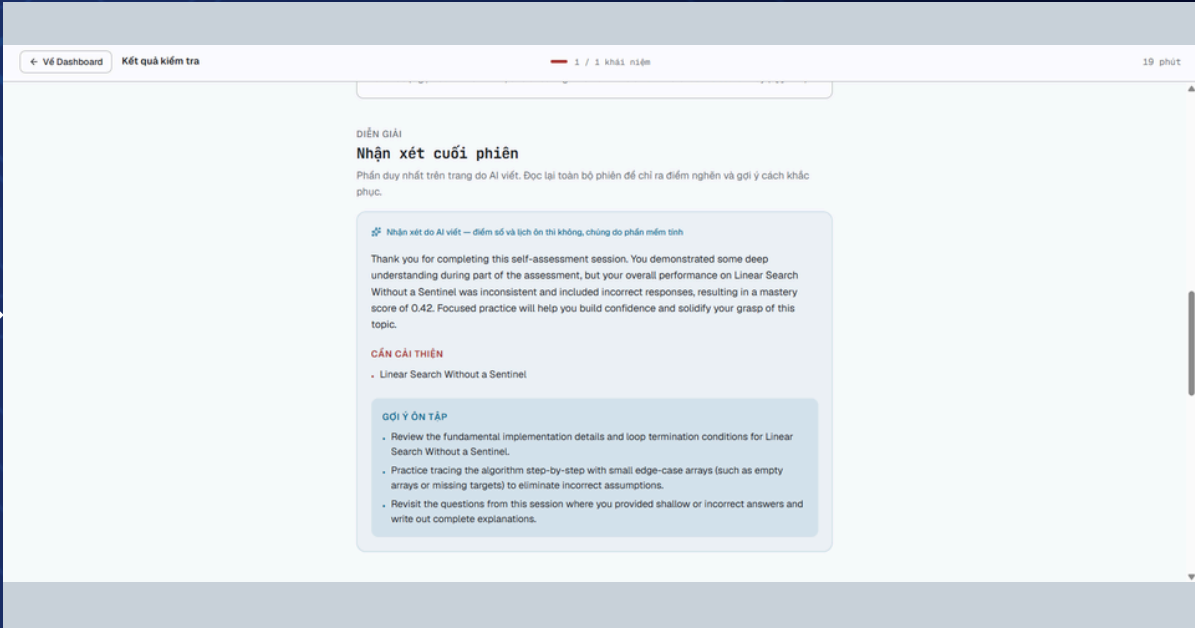
State machine decides: go deeper, probe further, or move on



After N turns → mastery_score saved; next concept begins

KEY BEHAVIORS

- Max 3 turns per concept - all flow control is deterministic software logic, not AI
- If AI unavailable → auto-fallback to cached flashcards (self-grading)
- If mastery_score < 0.6 → Traceback auto-schedules weak prerequisites into next session
- Session auto-saves on exit; student can resume incomplete sessions



Session ends → AI generates natural-language summary report



NON-FUNCTIONAL REQUIREMENTS

PERFORMANCE

- Page load (FCP) $\leq 3s$
- Standard API $\leq 1s$
- AI responses $\leq 15s$
- BFS Traceback (≤ 50 nodes) $\leq 500ms$

SECURITY

- HTTPS (TLS) for all data transmission
- Passwords hashed with bcrypt
- Sessions managed via JWT (access + refresh token)
- HTTP security headers enforced via Helmet + CORS

USABILITY

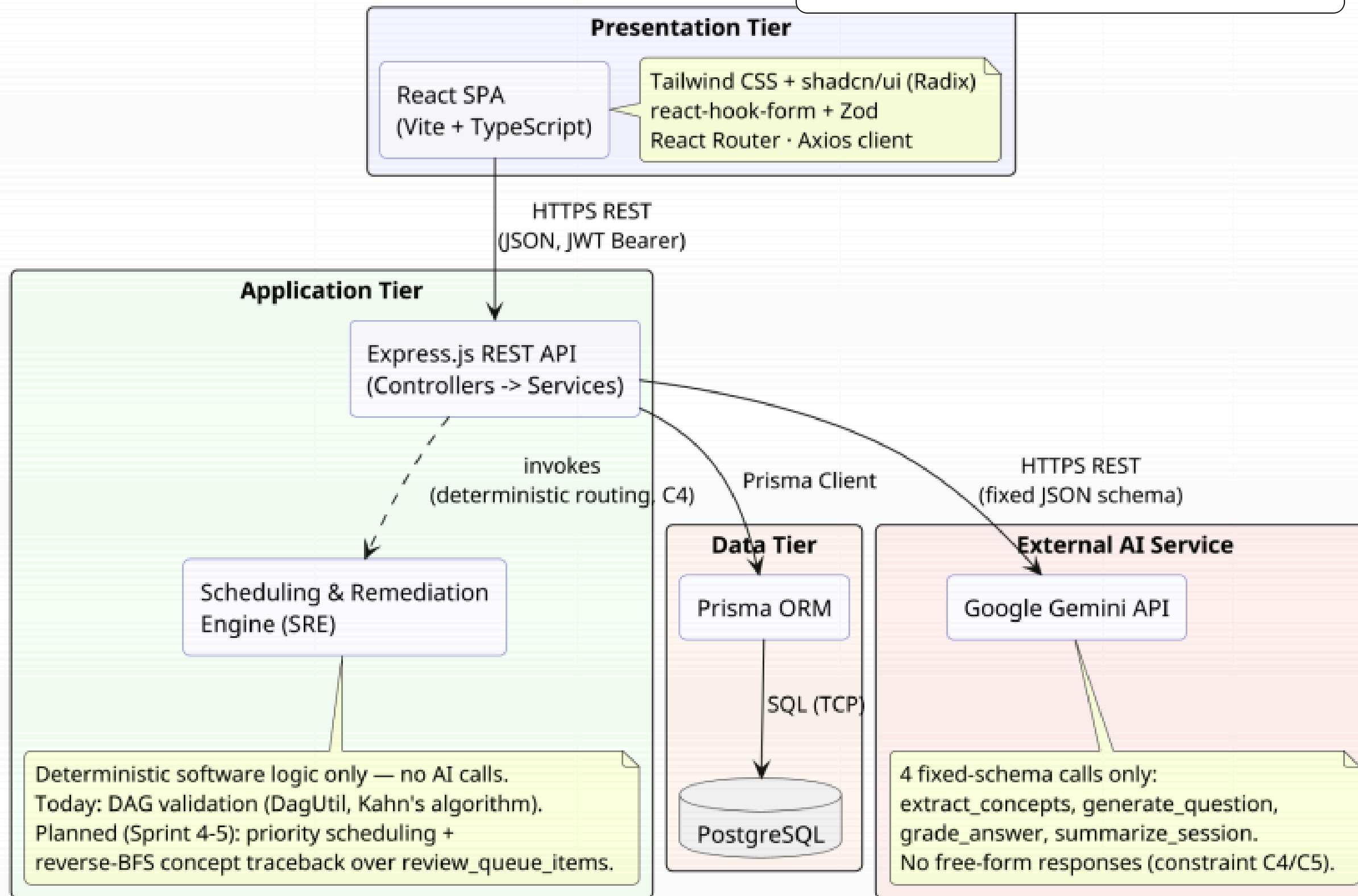
- Desktop-first ($\geq 1280px$)
- Chrome required
- Color contrast meets WCAG 2.1 AA ($\geq 4.5:1$)
- AI Examiner responds in the language of the uploaded document

AVAILABILITY & RELIABILITY

- 100% uptime during demo sessions
- AI failure $\rightarrow$ auto-fallback to cached flashcards
- Invalid AI output $\rightarrow$ auto-retry (max 3 $\times$), then split document by heading
- Data saved at every step – no loss on tab close

MAINTAINABILITY

- ESLint + Prettier + Husky enforced across entire codebase
- Key thresholds configurable via env vars (MASTERY_THRESHOLD, MAX_INTERVIEW_TURNS, MAX_REMEDIATION_DEPTH)
- AI layer abstracted as a service – provider-agnostic



SYSTEM ARCHITECTURE

Tier	Responsibility
Presentation	UI rendering, client-side validation, routing
Application	Business logic, auth, orchestration, SRE
Data	Persistent storage (PostgreSQL via Prisma)
External	Gemini API — called only for narrow, schema-constrained extraction/generation/grading tasks



TECH-STACK

FRONTEND

- React (Vite)
- TypeScript
- Tailwind CSS v4
- shadcn/ui
- React Router
- Axios

BACKEND

- Node.js
- Express.js
- TypeScript
- ZOD
- JWT + bcryptjs
- Multer
- Helmet + CORS

DATABASE

- PostgreSQL
- Prisma ORM 7

EXTERNAL AI

- Google Gemini API



SOFTWARE TESTING

TEST ENVIRONMENT

- **Setup:** Local/Dev environment (Node.js, TypeScript).
- **Data:** Local Database & Mock Data (Mock AI).
- **Tools:** Web Browser (UI), Postman (API), Console (Debugging), Playwright(e2e)

TEST RESULTS

- **Execution:** Completed core Test Cases.
- **Status:** Most Happy Paths Passed. Minor bugs and edge cases are fixed.
- **Assessment:** The system is generally stable and meets the initial requirements.

TEST PLAN

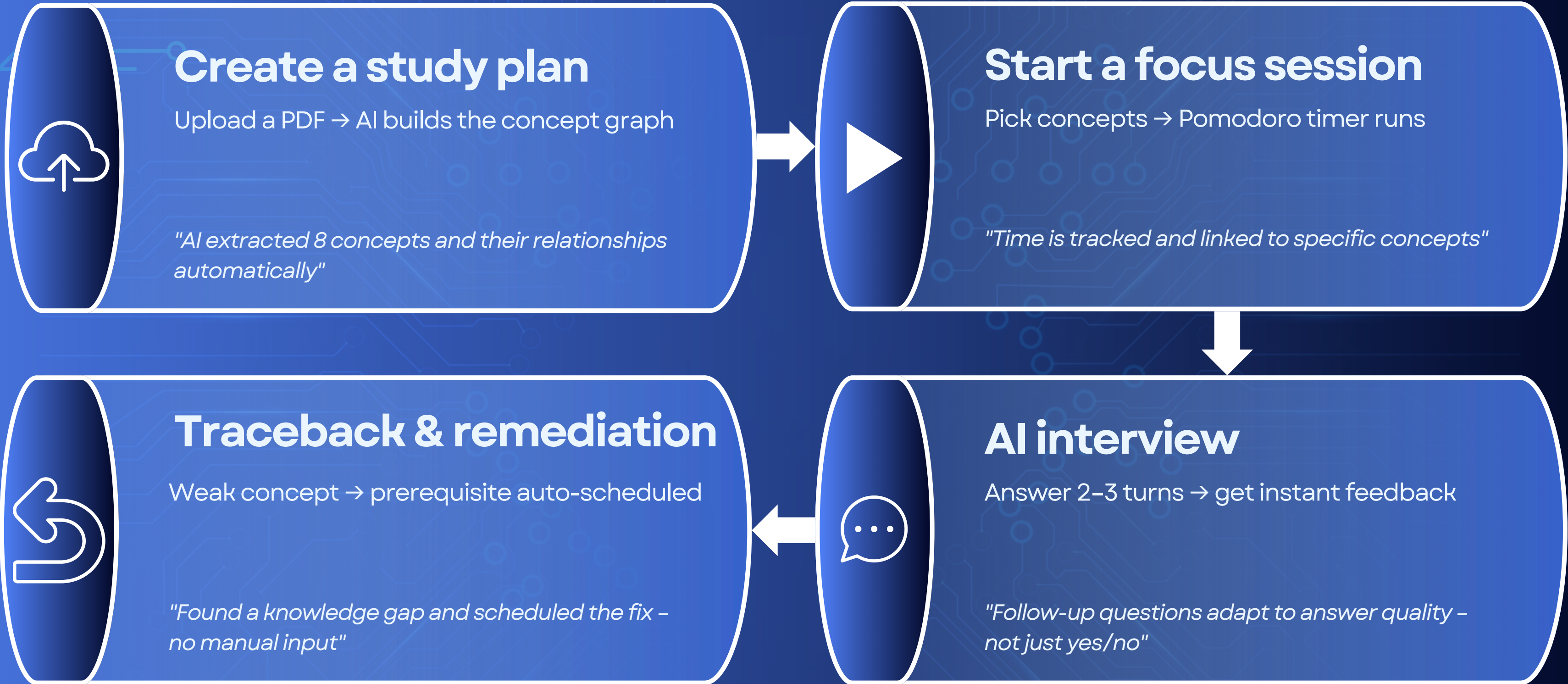
- **Scope:** Testing all system Use Cases (UCs), include Authenticator, Study Planner, AI Examiner, Dashboard, Focus Session
- **Strategy:** Functional Testing combined with API Testing and e2e testing.
- **Objective:** Ensure the core planning and data storage flows work seamlessly without critical errors.

AUTOMATED TESTING

- **E2E Testing:** Using Playwright to simulate user interactions and automatically test complete UI workflows from start to finish.
- **API Testing:** Using automated scripts (Postman/Jest) for quick API response validation.



KEY SCENARIO



Q&A

The background is a deep blue gradient. A large, semi-transparent light blue circle is centered in the background. Overlaid on this are abstract, flowing blue shapes that resemble liquid or smoke. Faint, light blue circuit board patterns and network diagrams (nodes connected by lines) are scattered across the background, particularly on the left and right sides.

**THANK
FOR LISTENING.**